LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 5
Student Name: V S SRAVAN M
Roll No.: A24126552268

---

1. TITLE

Working with Node.js Modules (CommonJS, Built-in, and Third-Party Packages)

---

2. AIM

To explore and implement Node.js module systems, including CommonJS (require/module.exports), core built-in modules (os, crypto), and third-party package management using npm (cowsay).

---

3. OBJECTIVE

The objectives of this experiment are:

- To understand modular architecture in server-side Node.js applications.
- To create and export custom reusable modules using CommonJS syntax (module.exports, require).
- To utilize core Node.js built-in modules (os for system diagnostics, crypto for secure UUID generation).
- To manage dependencies using npm, initializing package.json, and installing external packages.
- To execute Node.js scripts across different module environments and analyze terminal outputs.

---

4. THEORY

Modularity is a fundamental principle of modern software engineering that divides complex programs into independent, interchangeable modules. In Node.js, modules encapsulate functionality, manage dependencies, and expose public APIs while maintaining private internal implementations.

Theory of Node.js Module Architecture:

- **CommonJS Specification:** The traditional Node.js module system utilizing require() for synchronous dependency loading and module.exports for defining public interfaces.
- **Core Built-in Modules:** Modules compiled directly into the Node.js binary (such as os, crypto, fs, path) requiring no external installation.
- **npm Ecosystem:** The Node Package Manager (npm) coordinates third-party open-source packages declared in package.json and installed into node_modules.
- **Cryptographic Primitives:** The crypto module provides cryptographic functionality, including cryptographically secure pseudorandom UUID generation.
- **System Diagnostics:** The os module provides operating-system-related utility methods and system memory metrics.

---

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 5(week -5)/Src and Documentation.
2. In Src/CommonJS, implement message.cjs and mod.cjs demonstrating module.exports and require().
3. In Src/InBuilt, implement inbuilt.js utilizing core modules os (platform, free memory) and crypto (random UUID).
4. In Src/ThirdParty, initialize package.json and install third-party package cowsay via npm.
5. In others.js, require and execute cowsay with custom parameters and ASCII art rendering.
6. Execute each script via Node.js in the terminal and verify system recon, crypto ID generation, and cowsay output.

---

6. PROGRAM

CommonJS/message.cjs

```
const msg = ()  => {console.log("Hello from CommonJS module!");};
const add = (a,b) => {return a + b;};

module.exports = {msg, add};
```

CommonJS/mod.cjs

```
const mod1 = require('./message.cjs');
mod1.msg();
console.log(mod1.add(5, 10));
```

InBuilt/inbuilt.js

```
// hacker.mjs

// 1. Import the built-in modules
import os from 'os';
import crypto from 'crypto';

console.log("--- SYSTEM RECON ---");

// 2. Use 'os' to spy on your own computer
console.log("Operating System:", os.platform());
// Memory is in bytes, so we divide by 1024 twice to get Megabytes
console.log("Free RAM:", Math.round(os.freemem() / 1024 / 1024), "MB");

console.log("--- ENCRYPTION ---");

// 3. Use 'crypto' to generate a secure, random ID
const secretId = crypto.randomUUID();
console.log("Your Secret Hacker ID:", secretId);
```

ThirdParty/others.js

```
const cowsay = require('cowsay');
console.log(cowsay.say({
    text : "I am officially a Node.js developer now 🗿",
    e:"-O",
    T:"U "
}));
```

---

7. CODE EXPLANATION

Module / Construct	Explanation
`module.exports`	Special object used to expose functions or variables from a CommonJS module.
`require(path)`	Built-in function used to synchronously import modules in CommonJS.
`import os from 'os'`	Imports the core operating system diagnostic module in modern JS syntax.
`import crypto from 'crypto'`	Imports cryptographic services module for secure hashing and token generation.
`os.platform()`	Returns a string identifying the operating system platform (e.g. 'win32').
`os.freemem()`	Returns the amount of free system memory in bytes.
`crypto.randomUUID()`	Generates a cryptographically strong random RFC 4122 version 4 UUID.
`package.json`	Manifest file listing project metadata, scripts, and npm dependencies.
`cowsay.say({...})`	Third-party npm library method rendering ASCII character speech bubbles.

---

8. TEST CASES AND EXECUTION DATA

The following test suite was executed in Node.js runtime to verify module imports, built-ins, and npm packages:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	CommonJS Function Invocation	Execute node mod.cjs	Prints 'Hello from CommonJS module!' and computes sum 15	**Pass**
TC-02	OS Platform Identification	Execute node inbuilt.js	Correctly detects operating system platform as 'win32'	**Pass**
TC-03	Memory Calculation	Execute node inbuilt.js	Accurately calculates and displays free RAM in megabytes	**Pass**
TC-04	UUID Generation	Execute node inbuilt.js	Generates a valid 36-character hexadecimal RFC 4122 v4 UUID	**Pass**
TC-05	Third-Party Package Execution	Execute node others.js	Renders the cowsay speech bubble ASCII graphic successfully	**Pass**

---

9. OUTPUT

Output: Node.js Terminal Execution Output

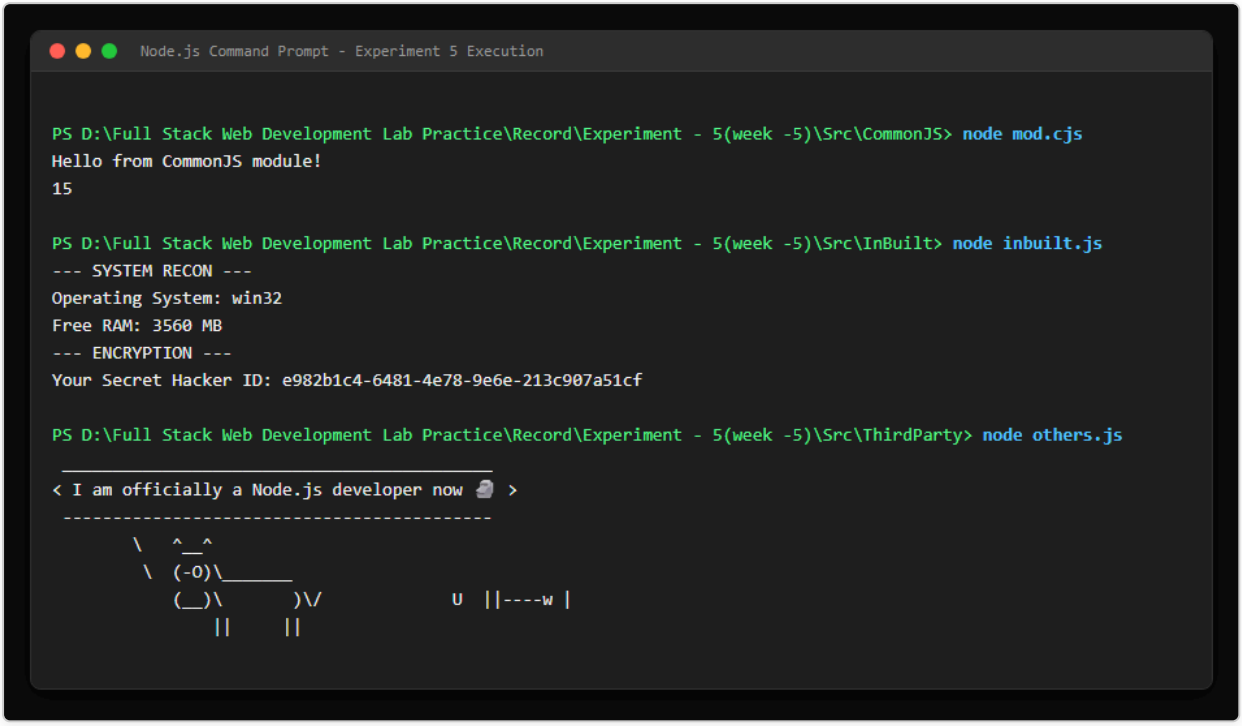

Figure 1: Terminal output displaying CommonJS module execution, system recon diagnostics, and cowsay ASCII art.

---

10. RESULT

Thus, Node.js module systems including CommonJS, built-in core modules (os, crypto), and third-party npm package integration were successfully implemented, executed, and verified under Node.js runtime environment.

Submission Package Structure:

```
Experiment - 5(week -5)/
├── Documentation/
│   ├── Experiment_5.docx
│   └── Experiment_5.pdf
├── Screenshot/
│   └── Screenshot 2026-09-01 101500.png
└── Src/
    ├── CommonJS/
    │   ├── message.cjs
    │   ├── mod.cjs
    │   └── package.json
    ├── InBuilt/
    │   └── inbuilt.js
    └── ThirdParty/
        ├── others.js
        └── package.json
```